Listing 11.2 Register routers with a `NinjaAPI` object and use it to create URLs

```
# RiffMates/RiffMates/urls.py
...
from ninja import NinjaAPI

from home.api import router as home_router

api = NinjaAPI(version="1.0")
api.add_router("/home/", home_router)

urlpatterns = [
    # ...
    path("api/v1/", api.urls),
]
```

Import the Router from the RiffMates/home/api.py.

Create a single NinjaAPI object.

Register the home app's router with NinjaAPI.

Add the API endpoints as URLs in your Django project.

I've made a couple of choices here that aren't necessary yet but help future-proof the API. First, when importing `router` from `home.api`, I alias it as `home_router`. If you guessed that I plan on adding api.py files to several apps, you'd be right. Second, I used the `version` argument when constructing the `NinjaAPI` object. This isn't required if you only have one API. If you wish to have multiple `NinjaAPI` objects, they must have unique version names. As I like to version my APIs anyway, I've done this from the start.

The `api.urls` property works similarly to `include()` in the `django.urls` module. All the API endpoints that are registered with the `NinjaAPI` object get included underneath the declared path. I've chosen `api/v1/` as my base path, first to namespace all the URLs as part of the API and second to include versioning information to future-proof any breaking changes.

That's it. You've got an API. Granted, it only has one endpoint, but it is an API. Fire up your development server and visit http://localhost:8000/api/v1/home/ to see it in action. Remember that Ninja's output is JSON, so it has a MIME type of `application/json`. Depending on what browser or tool you are using to hit the API, you may see different things. Firefox has a great JSON analysis tool built in. If you're using Firefox, you will see three tabs: the Content tab (RiffMates Rocks!), a Raw Data tab, and a Headers tab. For our home endpoint, the content and the raw data are the same: the serialized string from our view. When you move to more complex data structures, the content tab parses the JSON and makes it easier to deal with. For example, objects and lists can be collapsed, and you can apply filters. If you're using Chrome or one of the many Chrome-based browsers, you don't get all this fancy stuff—you only get the raw data. There are add-ons for Chrome that have similar features to Firefox's interface.

When dealing with APIs, the `curl` command-line tool (https://curl.se/) can be quite useful. This tool speaks numerous protocols, including HTTP, and can fetch any URL you give it. If you're interacting with a REST API that uses JSON or XML, doing so on the command line is sometimes easier. There are also dedicated tools, like `postman` (https://www.postman.com/product/rest-client/), if you prefer a GUI approach.